

SAVCNG ExcelDNA - Documentación Técnica Completa

Versión: 2.3.0.0

Última Actualización: 11 de julio de 2026

Estado: En Desarrollo Continuo

Tabla de Contenidos

1. Descripción General
2. Arquitectura del Proyecto
3. Componentes Principales
4. Sistema de Validaciones Detallado
 - 4.1 Validación de Decimales (Enteros)
 - 4.2 Validación de Catálogos
 - 4.3 Validación NS
 - 4.4 Validación de Formato Texto
 - 4.5 Validación de Bloqueo Dinámico
 - 4.6 Validación de Campos Vacíos (Blancos)
 - 4.7 Validación de Especifique por Palabras Clave
 - 4.8 Validación de Años (Fechas)
5. FrmValidaciones.cs - Análisis Profundo
6. Flujo de Trabajo
7. Especificaciones Técnicas
8. Patrones de Código y Mejores Prácticas
9. Troubleshooting y Casos Especiales

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegadas (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales
- Validar formato de texto: mayúsculas, sin espacios extra, caracteres válidos

- Crear reglas de bloqueo dinámico condicionadas a valores de otras celdas
- Soporte automático para matrices paralelas y evaluaciones fila por fila
- Mensajes de alerta personalizados que aparecen dinámicamente
- Detectar y resaltar campos vacíos en matrices con validación de obligatoriedad
- Detectar si el texto de un "Especifique" coincide con opciones del catálogo mediante palabras clave
- Validar que el año capturado se encuentre dentro de un rango histórico configurable

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/
├── MenuCenso.cs           (Punto de entrada y menú)
├── FrmValidaciones.cs     (Lógica principal de validaciones)
├── FrmValidaciones.Designer.cs (Diseño de formulario)
├── Properties/
│   └── AssemblyInfo.cs    (Metadatos del ensamblado)
├── bin/                   (Ejecutables compilados)
├── obj/                   (Artefactos compilados)
├── documentacion/
│   └── documentacion_savcng_YYYY-MM-DD.md (Documentación técnica)
```

Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

Componentes Principales

1. **MenuCenso.cs** - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
- **Manejo de Errores:** Captura excepciones durante la carga de archivos

Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

2. FrmValidaciones.cs - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

Variables de Estado Principales

```
private Excel.Workbook _libroCenso;           // Archivo de Excel cargado
private Excel.Range _rangoCapturado;        // Rango seleccionado por el usuario
private Excel.Range _rangoCatalogo;          // Rango de opciones del catálogo
private List<Excel.Range> _pendientesCatalogo;
private Dictionary<string, string> _preguntaPorRango;
```

Sistema de Validaciones Detallado

4.1 Validación de Decimales (Enteros)

Objetivo: Prevenir que el usuario capture números con decimales o texto cuando la celda solo admite enteros.

Tipo: Data Validation - Custom (Restrictiva)

Restricción: Sí (lanza ventana emergente si se viola)

Motor: `=O(ESBLANCO()); Y(ESNUMERO()); TRUNCAR(=valor))`

Cambio v2.3.0: Migrado de Formato Condicional (solo visual) a **Data Validation restrictiva**. El usuario ya **no puede** ingresar decimales ni texto; el sistema lanza un mensaje de alerta antes de aceptar el dato.

Fórmula:

```
=O(ESBLANCO(A1); Y(ESNUMERO(A1); TRUNCAR(A1)=A1))
```

La protección `ESBLANCO()` evita que la validación rechace celdas aún vacías en el momento de la inyección.

Mensaje de Error:

Título: Captura Inválida (Solo Enteros)
Mensaje: El formato de esta celda no admite números con decimales ni texto.
Por favor, introduce únicamente un número entero (Ej: 1, 15, 100).

Implementación:

```
Excel.Range primeraCelda = (Excel.Range)_rangoCapturado.Cells[1, 1];
string direccion = primeraCelda.get_Address(false, false,
Excel.XlReferenceStyle.xlA1, false);

string formulaDecimales =
    $"=0(ESBLANCO({direccion}){separador}Y(ESNUMERO({direccion})
{separador}TRUNCAR({direccion})={direccion}))";

_rangoCapturado.Validation.Add(
    Excel.XlDVType.xlValidateCustom,
    Excel.XlDValertStyle.xlValidAlertStop,
    Excel.XlFormatConditionOperator.xlBetween,
    formulaDecimales,
    Type.Missing);

_rangoCapturado.Validation.IgnoreBlank = true;
_rangoCapturado.Validation.ShowError = true;
_rangoCapturado.Validation.ErrorTitle = "Captura Inválida (Solo Enteros)";
_rangoCapturado.Validation.ErrorMessage = "El formato de esta celda no admite
números con decimales ni texto...";
```

4.2 Validación de Catálogos

Objetivo: Crear listas desplegables (Data Validation) con opciones predefinidas.

Tipo: Data Validation - Lista

Restricción: Sí (solo permite valores de la lista)

Motor: Rango de opciones o entrada manual

Características:

- Entrada manual de opciones separadas por comas (ej: "1,2,9" o "X")
- Selección de celdas con tres modos: Solo Números, Texto Exacto, Referencia de Rango
- Soporte para celda única, celdas combinadas y rangos múltiples

- Limpieza automática de caracteres innecesarios

Flujo de Configuración:

1. Sistema pregunta: ¿Entrada manual o seleccionar celdas?
2. Si es manual: Ingresar opciones separadas por comas
3. Si es celda única/combinada: Elegir entre Números, Texto Exacto o Referencia
4. Si es rango múltiple: Mismas tres opciones
5. Sistema reemplaza comas por el separador local del sistema antes de aplicar

4.3 Validación NS (No Aplica)

Objetivo: Permitir números ≥ 0 o valor "NS", con alerta dinámica cuando se registra NS.

Tipo: Data Validation - Custom + Fórmula de alerta

Restricción: Sí (solo permite números ≥ 0 o "NS")

Motor: `=O(Y(ESNUMERO()); >=0); ="NS")`

Cambio v2.3.0: El destino del mensaje de alerta y su texto son ahora **configurables por el usuario** mediante dos InputBox previos, en lugar de ser fijos en la fila siguiente.

Flujo de Configuración:

1. **Paso 1:** Usuario selecciona la celda o rango donde aparecerá el mensaje de alerta
2. **Paso 2:** Usuario escribe (o confirma) el texto del mensaje

Fórmula de Validación (español):

```
=O(Y(ESNUMERO(A1); A1>=0); A1="NS")
```

Fórmula de Alerta (inglés universal):

```
=IF(SUM(COUNTIF($A$1:$A$10,"NS"), COUNTIF($B$1:$B$10,"NS"))>0, "Alerta...", "")
```

Formato del Mensaje: Arial 9pt, Negrita, Color Dorado RGB(191, 143, 0), alineación izquierda.

Implementación clave:

```
// Motor de conteo universal: itera áreas del rango (soporte rangos no contiguos)
for (int i = 1; i <= _rangoCapturado.Areas.Count; i++)
{
    Excel.Range area = (Excel.Range)_rangoCapturado.Areas[i];
    string addrAbs = area.get_Address(true, true, Excel.XlReferenceStyle.xlA1,
```

```

false);
partesCountIf.Add($"COUNTIF({addrAbs},\"NS\")");
}
string sumaInner = string.Join(",", partesCountIf);
string formulaFinalAlerta = $"=IF(SUM({sumaInner})>0, \"{textoAlerta}\", \"\")";
rangoAlerta.Formula = formulaFinalAlerta; // .Formula = INGLÉS universal

```

4.4 Validación de Formato Texto

Objetivo: Garantizar texto en mayúsculas, sin espacios extra y sin caracteres especiales.

Tipo: Data Validation - Custom con Rango Auxiliar

Restricción: Sí (previene entrada inválida)

Motor: Fórmula binaria en columna auxiliar + `= {celdaAux}=1`

Cambio v2.3.0 (Arquitectura Auxiliar): Se reemplazó la fórmula simple `IGUAL + ESPACIOS` por un motor de **Whitelist de caracteres** con rango auxiliar dinámico. La Data Validation ahora apunta a una celda auxiliar que evalúa todas las reglas.

Reglas Implementadas:

- ✓ Solo mayúsculas (A-Z, Ñ, acentos: Á É Í Ó Ú Ü)
- ✓ Solo dígitos (0-9)
- ✓ Espacios simples (sin dobles espacios, sin espacios al inicio/final)
- ✓ Sin caracteres especiales (signos de puntuación, símbolos)

Whitelist de Caracteres Permitidos:

0123456789ABCDEFGHIJKLMNÑOPQRSTUVWXYZÁÉÍÓÚÜ (espacio)

Flujo de Configuración:

1. Usuario selecciona la columna auxiliar vacía (ej: columna CW)
2. Sistema construye el rango auxiliar alineado con las filas capturadas
3. Sistema inyecta fórmula binaria en la columna auxiliar
4. Sistema aplica Data Validation que apunta al auxiliar: `= {celdaAux}=1`

Fórmula Auxiliar (binaria, inglés universal):

```

=IF(OR(ISBLANK(A1),
    AND(EXACT(A1, UPPER(A1)),
        LEN(A1)=LEN(TRIM(A1))),
    SUMPRODUCT(--
        ISNUMBER(FIND(MID(A1,ROW(INDIRECT("1:"&MAX(1,LEN(A1))))),1),"0123456789ABC..."))=LEN

```

```
(A1)
)), 1, 0)
```

- Retorna **1** si la celda es válida o está vacía
- Retorna **0** si viola alguna regla

Fórmula de Data Validation (ultra ligera):

```
={celdaAux}=1
```

Bypass del error 0x800A03EC:

```
// Si la primera celda está vacía, se coloca "A" temporalmente
// para que Excel acepte la inyección de la fórmula de validación
object valorOriginal = primeraCeldaCap.Value2;
bool estabaVacía = (valorOriginal == null ||
string.IsNullOrEmpty(valorOriginal.ToString()));
if (estabaVacía) { primeraCeldaCap.Value2 = "A"; }

// ... aplicar validación ...

if (estabaVacía) { primeraCeldaCap.Value2 = null; } // Restaurar
```

Mensaje de Error:

```
Título: Formato de texto inválido
Mensaje: El texto capturado debe cumplir estrictamente las siguientes reglas:
    • Todo en MAYÚSCULAS.
    • Sin dobles espacios ni espacios a las orillas.
    • SOLO LETRAS (incluye Ñ y acentos) Y NÚMEROS. No se permiten caracteres
especiales.
```

4.5 Validación de Bloqueo Dinámico

Objetivo: Crear reglas de bloqueo condicional basadas en valores de otras celdas, con soporte para matrices paralelas.

Tipo: Data Validation - Custom + Formatos Condicionales

Restricción: Sí (bloquea entrada si condición no se cumple)

Motor: **CONTAR.SI()** con detección inteligente de modo fila-por-fila o global

Características:

- Detección automática de matrices paralelas (mismas filas)
- Regla de celda vacía configurable (Paso 3.5)
- Auditoría de coexistencia con formatos previos
- Tres estados visuales: Gris Bloqueado, Normal, Azul Obligatorio
- Soporte para 6 operadores: `=`, `<>`, `>`, `<`, `>=`, `<=`

Flujo de Configuración (4 pasos):

1. **Paso 1:** Seleccionar rango/celda de condición
2. **Paso 2:** Elegir operador lógico (`=`, `<>`, etc.)
3. **Paso 3:** Ingresar valor del criterio
4. **Paso 3.5:** Elegir comportamiento con celda vacía:
 - SÍ: Si la condición está en blanco, la celda se desbloquea
 - NO: Estricto, si está en blanco se bloquea
5. **Paso 4:** Opcional — Resalte azul cuando está desbloqueada pero vacía

Fórmulas generadas según configuración:

Con celda vacía = Sí:

```
Validación: =O(ESBLANCO(Cond); CONTAR.SI(Cond; "=Valor")>0)
Sombreado:  =Y(ESBLANCO(Cond)=FALSO; CONTAR.SI(Cond; "=Valor")=0)
```

Con celda vacía = No:

```
Validación: =CONTAR.SI(Cond; "=Valor")>0
Sombreado:  =CONTAR.SI(Cond; "=Valor")=0
```

Jerarquía de Formatos Condicionales:

```
Regla 1 – Gris (Bloqueado):  StopIfTrue = true → si aplica, ignora regla 2
Regla 2 – Azul (Obligatorio): StopIfTrue = true → si aplica, ignora estilos base
```

Auditoría de Coexistencia:

```
if (_rangoCapturado.FormatConditions.Count > 0)
{
    // Pregunta al usuario si conservar o eliminar formatos previos
}
```



```
// Permite apilar bloqueo sobre validación de blancos  
}
```

Detección de dirección fila-por-fila:

```
if (esFilaPorFila)  
{  
    // Ancla columna, fila relativa: $A1  
    string addressAbsoluta = rangoCondicion.Cells[1, 1].Address; // "$A$1"  
    string[] partes = addressAbsoluta.Split('$');  
    dirCondicionLocal = "$" + partes[1] + partes[2]; // "$A1"  
}  
else  
{  
    dirCondicionLocal = rangoCondicion.Address; // "$A$1:$A$20"  
}
```

4.6 Validación de Campos Vacíos (Blancos)

Objetivo: Detectar y resaltar celdas vacías en filas que ya tienen datos en otras columnas.

Tipo: Fórmula de alerta + Formato Condicional

Restricción: No (solo resalta visualmente)

Motor: `SUMPRODUCT()` con álgebra booleana fila-por-fila

Características:

- Motor matricial dinámico (itera columna por columna)
- Alerta global con texto personalizado, color azul
- Formato condicional fila-por-fila con álgebra booleana
- Auditoría de coexistencia con validación de Bloqueo

Flujo de Configuración (2 pasos):

1. Seleccionar celda/rango destino del mensaje azul
2. Ingresar texto del mensaje de alerta

Fórmula de Alerta Global (inglés universal):

```
=IF(SUMPRODUCT(($M$52:$M$71<>"")+($N$52:$N$71<>""))*  
    (($M$52:$M$71="")+($N$52:$N$71=""))>0,  
    "Favor de revisar información faltante", "")
```

Fórmula de Formato Condicional (álgebra booleana):

```
=(M52="")*((($M52<>"") + ($N52<>"") + ...) > 0)
```

Implementación del Motor:

```
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range col = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = col.get_Address(true, true, Excel.XlReferenceStyle.xlA1,
false);
    listaActivadores.Add($"({dirColAbs}<>"")");
    listaVacios.Add($"({dirColAbs}=""")");
}
string formulaGlobal =
    $"=IF(SUMPRODUCT(({motorActivadores})*({motorVacios}))>0, \"{textoAlerta}\",
\"\")";
rangoDestino.Formula = formulaGlobal;
```

4.7 Validación de Especifique por Palabras Clave ★ (v2.3.0)

Objetivo: Detectar si el texto libre capturado en un campo "Especifique" coincide semánticamente con alguna de las opciones de un catálogo, alertando que posiblemente debería marcarse una opción en lugar de escribir texto libre.

Tipo: Motor Auxiliar Matricial + Formato Condicional + Alerta

Restricción: No (es informativa; resalta y alerta, no impide captura)

Motor: Diccionario de palabras clave con **SEARCH()** + **OR()** nativo de Excel

Contexto de Uso:

Catálogo:

- ☐ Transparencia y rendición de cuentas
- ☐ Educación y formación
- ☐ Salud pública

Especifique: "educacion basica"

- Sistema detecta coincidencia con "Educación y formación"
- Resalta esa opción en amarillo
- Muestra mensaje de alerta dorado

Flujo de Configuración (3 pasos):

1. **Paso 1:** Seleccionar las celdas del catálogo de opciones
2. **Paso 2:** Seleccionar la celda destino del mensaje de alerta (color dorado)
3. **Paso 3:** Seleccionar una celda vacía como punto de anclaje del motor auxiliar
 - Se necesita espacio de 2 columnas × N filas (N = cantidad de opciones del catálogo)

Fases de Implementación:

Fase 1 — Celda de Limpieza:

```
// Normaliza el texto del Especifique: quita acentos y pasa a minúsculas
string formulaLimpieza =
    $"=LOWER(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(" +
    $"
    {dirAzulAbsoluta},\"á\", \"a\"), \"é\", \"e\"), \"í\", \"i\"), \"ó\", \"o\"), \"ú\", \"u\")))"
    ;
celdaLimpiaAzul.Formula = formulaLimpieza;
```

Fase 2 — Motor de Palabras Clave (por cada opción del catálogo):

El sistema en C# procesa cada opción del catálogo:

1. Elimina contenido entre paréntesis (ej: "(transparencia)" → "")
2. Quita acentos y convierte a minúsculas
3. Divide por conectores: y/o, y, o, e, , , /
4. Filtra palabras con más de 2 caracteres (descarta artículos)
5. Construye fórmulas `ISNUMBER(SEARCH("palabra", celdaLimpia))`

```
string[] separadoresPalabras = { " y/o ", " y ", " o ", " e ", ",", "/" };
string[] palabrasClave = textoProcesado.Split(separadoresPalabras,
StringSplitOptions.RemoveEmptyEntries);

foreach (string palabra in palabrasClave)
{
    if (palabra.Trim().Length > 2)
        fragmentosSearch.Add($"ISNUMBER(SEARCH(\"{palabra.Trim()}\",
{dirLimpiaAbsoluta}))");
}

string formulaMatch = $"=OR({string.Join(",", fragmentosSearch)})";
celdaMatch.Formula = formulaMatch; // Retorna TRUE si hay coincidencia
```

Fase 3 — Traducción de Fórmulas (Truco Arquitectónico):

Para evitar el error `#¿NOMBRE?` al inyectar fórmulas en Excel con idioma español:

```
// Se inyecta la fórmula en inglés en la última celda de la hoja (XFD1048576)
Excel.Range celdaDummy = ws.Cells[1048576, 16384];
celdaDummy.Formula = formulaFormatCondIngles;
string formulaFormatCondLocal = celdaDummy.FormulaLocal; // Excel traduce
automáticamente
celdaDummy.Clear();
// Se usa la fórmula traducida para el FormatCondition
```

Fase 4 — Mensaje de Alerta:

```
// Cuenta cuántas celdas del motor tienen TRUE
string formulaMensajeFinal =
    $"=IF(COUNTIF({rangoResultados}, TRUE)>0," +
    $"\"Alerta: Revise el texto ingresado en el Especifique ya que podría existir "
+
    $"en las opciones resaltadas en amarillo\", \"\")";
rangoMensaje.Formula = formulaMensajeFinal;
```

Formato del Mensaje: Arial 9pt, Negrita, Color Dorado RGB(191, 144, 0).

Gestión de Memoria:

```
finally
{
    if (rangoCatalogo != null) Marshal.ReleaseComObject(rangoCatalogo);
    if (rangoMensaje != null) Marshal.ReleaseComObject(rangoMensaje);
    if (celdaMotor != null) Marshal.ReleaseComObject(celdaMotor);
}
```

Ventajas:

- ✓ Compatible con Excel español sin errores de localización
- ✓ Ignora acentos y mayúsculas en la comparación
- ✓ Ignora contenido entre paréntesis en las opciones del catálogo
- ✓ No bloquea la captura (es informativa)
- ✓ Libera objetos COM correctamente

4.8 Validación de Años (Fechas) ★ (v2.3.0)

Objetivo: Restringir la captura a años numéricos dentro de un rango histórico configurable, con soporte adicional para el código "NS".

Tipo: Data Validation - Custom

Restricción: Sí (lanza mensaje de error si el valor está fuera del rango)

Motor: `=0(ESPACIOS()="NS"; Y(ESNUMERO(); >=LimInf; <=LimSup))`

Características:

- Límites inferior y superior configurables por el usuario en tiempo de ejecución
- Acepta "NS" como valor válido además del rango de años
- Validación numérica de entradas antes de inyectar
- Prevención de fugas de memoria con `Marshal.ReleaseComObject`
- Desactiva y restaura `ScreenUpdating` para mejor rendimiento visual
- Manejo diferenciado de errores COM vs errores generales

Flujo de Configuración (2 pasos):

1. Usuario ingresa el **límite inferior** (ej: 1821, valor sugerido predeterminado)
2. Usuario ingresa el **límite superior** (ej: 2026, valor sugerido predeterminado)
3. Sistema valida que sean enteros y que inferior $\leq$ superior

Fórmula de Validación (español con separador dinámico):

```
=0(ESPACIOS(A1)="NS"; Y(ESNUMERO(A1); A1>=1821; A1<=2026))
```

Mensaje de Error:

Título: Validación de Consistencia

Mensaje: El valor ingresado debe ser un año válido de 4 dígitos entre {limiteInferior} y {limiteSuperior}, o el código 'NS'.

Implementación:

```
Excel.Validation objValidacion = null;
Excel.Range celdaInicial = null;

try
{
    excelApp.ScreenUpdating = false;

    objValidacion = _rangoCapturado.Validation;
    objValidacion.Delete();

    celdaInicial = (Excel.Range)_rangoCapturado.Cells[1, 1];
    string direccionRelativa = celdaInicial.get_Address(false, false,
        Excel.XlReferenceStyle.xlA1, Type.Missing, Type.Missing);
```

```

string formulaValidacion =
    $"=O(ESPACIOS({direccionRelativa})=\"NS\"{separador}\" +
    $"Y(ESNUMERO({direccionRelativa}){separador}\" +
    $" {direccionRelativa}>={limiteInferior}{separador}\" +
    $" {direccionRelativa}<={limiteSuperior}))";

objValidacion.Add(Excel.XlDVType.xlValidateCustom,
    Excel.XlDAlertStyle.xlValidAlertStop,
    Excel.XlFormatConditionOperator.xlBetween,
    formulaValidacion, Type.Missing);

objValidacion.IgnoreBlank = true;
objValidacion.ShowError = true;
objValidacion.ErrorTitle = "Validación de Consistencia";
objValidacion.ErrorMessage = $"El valor ingresado debe ser un año válido entre "
+
    $" {limiteInferior} y {limiteSuperior}, o el código
'NS'.";
}
finally
{
    if (celdaInicial != null) Marshal.ReleaseComObject(celdaInicial);
    if (objValidacion != null) Marshal.ReleaseComObject(objValidacion);
    excelApp.ScreenUpdating = true;
}

```

FrmValidaciones.cs - Análisis Profundo

5.1 Método: btnCapturarRango_Click

Responsabilidad: Capturar el rango seleccionado por el usuario y detectar el contexto de pregunta.

```

if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;
    string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
    lblPregunta.Text = "Pregunta detectada: " + pregunta;
    lblRangoSeleccionado.Text = "Rango seleccionado: " +
        _rangoCapturado.Address.Replace("$", "");

    MessageBox.Show(this, "Se capturó correctamente el rango: " +
        _rangoCapturado.Address, "Captura exitosa",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
}

```

Nota v2.3.0: Todos los `MessageBox.Show` pasaron a usar `this` como primer parámetro para mantener la ventana como propietaria del diálogo.

5.2 Método: btnAplicar_Click - Estructura General v2.3.0

```
btnAplicar_Click()
├─ Validación previa: _rangoCapturado != null
├─ if (chkDecimales.Checked)
│   └─ Validación de Enteros – Data Validation restrictiva ★ cambiada
├─ else if (chkCatalogos.Checked)
│   └─ Validación de Catálogos – Manual o Seleccionar celdas
├─ else if (chkNS.Checked)
│   └─ Validación NS – Destino y texto configurables ★ cambiada
├─ else if (chkFormatoTexto.Checked)
│   └─ Validación Formato Texto – Motor auxiliar + Whitelist ★ cambiada
├─ else if (chkBloqueo.Checked)
│   └─ Validación de Bloqueo Dinámico – 4 pasos + auditoría
├─ else if (chkBlancos.Checked)
│   └─ Validación de Campos Vacíos – Motor SUMPRODUCT + auditoría
├─ else if (chkEspClave.Checked)
│   └─ Validación Especifique por Palabras Clave ★ NUEVA
│       ├── Fase 1: Celda de limpieza (normalización)
│       ├── Fase 2: Motor de palabras clave por opción
│       ├── Fase 3: Traducción de fórmulas (celda dummy)
│       └─ Fase 4: Mensaje de alerta global
└─ else if (chkFechas.Checked)
    └─ Validación de Años ★ NUEVA
        ├── Captura de límite inferior y superior
        ├── Validación numérica de entradas
        ├── Fórmula: O(ESPACIOS="NS"; Y(ESNUMERO; >=; <=))
        └─ Gestión de memoria y ScreenUpdating
```

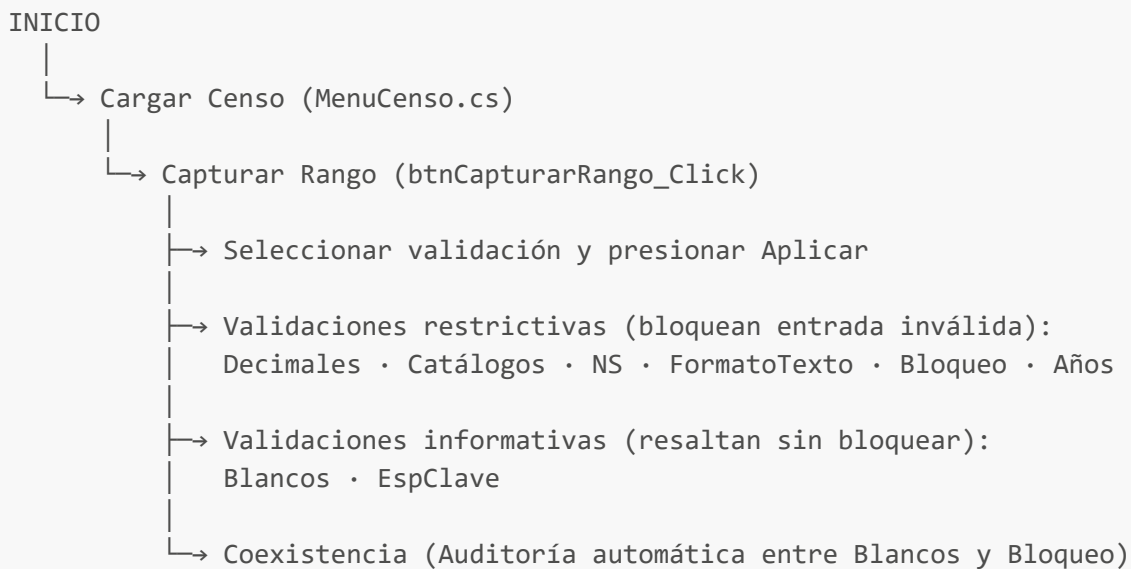
5.3 Métodos CheckedChanged

Todos los checkboxes siguen el mismo patrón de validación previa:

```
private void chkXxx_CheckedChanged(object sender, EventArgs e)
{
    if (chkXxx.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show(this, "Primero carga un censo y captura un rango.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            chkXxx.Checked = false;
        }
    }
}
```

Métodos existentes: `chkCatalogos_CheckedChanged`, `chkNS_CheckedChanged`, `chkBlancos_CheckedChanged`, `chkFechas_CheckedChanged`.

Flujo de Trabajo v2.3.0



Especificaciones Técnicas

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8
Versión C#	7.3

Aspecto	Requisito
Versión Excel	2013 o superior
SO	Windows 7/8/10/11 (32 o 64 bits)
Memoria RAM Mínima	4 GB
Bitness	Mismo que Office instalado (32 o 64)

Dependencias

```
Microsoft.Office.Interop.Excel (versión 15.0+)
ExcelDna.Integration
System.Windows.Forms
System.Drawing
System.Text.RegularExpressions
System.Reflection
System.Collections.Generic
System.Runtime.InteropServices
```

Internacionalización

Español	Inglés	Función
ESNUMERO()	ISNUMBER()	Verificar si es número
TRUNCAR()	TRUNC()	Truncar decimales
CONTAR.SI()	COUNTIF()	Contar con condición
SUMPRODUCT()	SUMPRODUCT()	Suma de productos
SI()	IF()	Condicional
O()	OR()	O lógico
Y()	AND()	Y lógico
MAYUSC()	UPPER()	Convertir a mayúsculas
ESPACIOS()	TRIM()	Remover espacios extra
IGUAL()	EXACT()	Comparación exacta
LARGO()	LEN()	Largo de texto
ESBLANCO()	ISBLANK()	Verificar si está vacío
BUSCAR()	SEARCH()	Buscar texto

Separador dinámico:

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();
// España/Hispanoamérica: ";" | USA: ","
```

Rendimiento v2.3.0

Operación	Tiempo Estimado
Captura de rango	Inmediato
Validación Decimales	1-2 seg
Validación Catálogos	2-3 seg
Validación NS	3-5 seg
Validación Formato Texto	2-4 seg
Validación Bloqueo completa	10-15 seg
Validación Blancos matriz 50×7	5-8 seg
Validación EspClave (catálogo 20 opciones)	5-10 seg
Validación Años	1-2 seg

Límites Conocidos

- Máximo rango: 1,048,576 filas × 16,384 columnas
- Máximo opciones catálogo: ~256 opciones
- Máximo formatos condicionales: 3 reglas simultáneas recomendadas
- Máximo texto mensaje: 255 caracteres
- Máximo opciones EspClave: ~50 opciones (por rendimiento)
- Máximo columnas matriz Blancos: ~50 (límite práctico)

Patrones de Código y Mejores Prácticas

Pattern 1: MessageBox con propietario

```
// CORRECTO v2.3.0: Pasa "this" para que el diálogo sea modal al formulario
MessageBox.Show(this, "Mensaje", "Título", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
```

```
// ANTERIOR: Sin propietario (podía quedar detrás del formulario)
// MessageBox.Show("Mensaje", "Título", ...);
```

Pattern 2: Casting Explícito (COM Interop)

```
// CORRECTO: Casting explícito antes de acceder a propiedades
Excel.Range primeraCelda = (Excel.Range)_rangoCapturado.Cells[1, 1];
string direccion = primeraCelda.Address;

// INCORRECTO: Falla con "'System.__ComObject' no contiene definición para
'Address'"
// string direccion = _rangoCapturado.Cells[1, 1].Address;
```

Pattern 3: Gestión de Memoria COM

```
// CORRECTO: Liberar objetos COM en bloque finally
Excel.Range obj = null;
try
{
    obj = (Excel.Range)excelApp.InputBox(..., 8);
    // ... usar obj ...
}
finally
{
    if (obj != null) System.Runtime.InteropServices.Marshal.ReleaseComObject(obj);
}
```

Pattern 4: Fórmulas Universales en INGLÉS

```
// CORRECTO: .Formula acepta inglés en cualquier Excel regional
rangoDestino.Formula = "=IF(COUNTIF($A$1:$A$10,\"NS\")>0,\"Alerta\",\"\")";

// INCORRECTO: .FormulaLocal requiere traducción manual
// rangoDestino.FormulaLocal =
// "SI(CONTAR.SI($A$1:$A$10;\"NS\")>0;\"Alerta\";\"\")";
```

Pattern 5: Álgebra Booleana Universal

```
// CORRECTO: Multiplicación lógica (compatible con Excel español/inglés)
string formula = $"=({celdaVacía}=\"\")*(({sumaActivadores})>0)";
```

```
// ALTERNATIVA CON AND() puede fallar en configuraciones locales específicas
```

Pattern 6: Bypass de Error 0x800A03EC

```
// Cuando la primera celda está vacía, Excel rechaza inyectar la DV.  
// Solución: poner valor temporal, inyectar, restaurar.  
bool estabaVacía = (primeraCeldaCap.Value2 == null);  
if (estabaVacía) primeraCeldaCap.Value2 = "A";  
// ... inyectar validación ...  
if (estabaVacía) primeraCeldaCap.Value2 = null;
```

Pattern 7: Traducción de Fórmulas vía Celda Dummy

```
// Para obtener la fórmula en el idioma local del usuario sin escribirla  
manualmente:  
Excel.Range celdaDummy = ws.Cells[1048576, 16384]; // XFD1048576 (última celda)  
celdaDummy.Formula = formulaEnIngles;  
string formulaLocal = celdaDummy.FormulaLocal; // Excel traduce  
celdaDummy.Clear();  
// Usar formulaLocal en FormatConditions.Add(...)
```

Troubleshooting y Casos Especiales

Problema 1: Error 'System.__ComObject' no contiene definición para 'Address'

Causa: Acceder a propiedades de un objeto COM sin hacer casting explícito.

Solución:

```
Excel.Range celda = (Excel.Range)rango.Cells[1, 1];  
string dir = celda.Address; // ✓
```

Problema 2: Error 0x800A03EC al inyectar Data Validation

Causa: La primera celda del rango está vacía al momento de crear la validación.

Solución: Aplicar el bypass temporal de valor "A" antes de la inyección.

Problema 3: Fórmula muestra #¿NOMBRE? en Excel español

Causa: Se inyectó una función en inglés mediante `.FormulaLocal`.

Solución: Usar siempre `.Formula` (acepta inglés universal) o el truco de la celda dummy para traducción automática.

Problema 4: Formato condicional desaparece al aplicar segunda validación

Causa: Se llama a `FormatConditions.Delete()` sin preguntar al usuario.

Solución: Implementar la auditoría de coexistencia antes de borrar.

Problema 5: `SUMPRODUCT` devuelve `#N/D`

Causa: Los vectores dentro de `SUMPRODUCT` tienen tamaños distintos.

Solución: Iterar columna-por-columna construyendo vectores del mismo tamaño en lugar de usar un único `SUMPRODUCT` sobre toda la matriz.

Problema 6: `EspClave` muestra `#¿NOMBRE?` en el formato condicional

Causa: `FormatConditions.Add` no acepta funciones en inglés directamente en Excel español.

Solución: Usar el truco de la celda dummy (`XFD1048576`) para obtener la fórmula traducida antes de aplicarla al formato condicional.

Resumen de Cambios v2.3.0

✦ Nuevas Validaciones

1. Validación de Especifique por Palabras Clave (`chkEspClave`)

- Motor auxiliar matricial de 2 columnas
- Normalización: elimina acentos, ignora paréntesis
- Segmentación por conectores lógicos
- Resaltado de opciones coincidentes en amarillo
- Mensaje de alerta en color dorado
- Gestión de memoria con `Marshal.ReleaseComObject`

2. Validación de Años (`chkFechas`)

- Límites inferior y superior configurables
- Acepta "NS" como valor válido
- Validación de consistencia numérica de entradas
- `ScreenUpdating = false` durante la inyección
- Gestión de memoria con `Marshal.ReleaseComObject`

🔧 Validaciones Mejoradas

3. Validación de Decimales (antes: FormatCondition → ahora: Data Validation)

- Ahora es **restrictiva** (impide la entrada en lugar de solo colorear)
- Fórmula con **ESBLANCO()** para protección de celdas vacías
- Mensaje de error personalizado con guía al usuario

4. Validación NS

- Destino del mensaje de alerta ahora es **seleccionable** por el usuario
- Texto del mensaje ahora es **editable** por el usuario
- Motor de conteo migrado a inglés universal (**.Formula**)

5. Validación de Formato Texto

- Reemplaza fórmula **IGUAL + ESPACIOS** por **Whitelist de caracteres**
- Columna auxiliar dinámica alojada en columna indicada por el usuario
- Soporte para Ñ y vocales acentuadas (Á É Í Ó Ú Ü)
- Bypass del error **0x800A03EC** con valor temporal

6. **MessageBox.Show** con **this**

- Todos los mensajes usan **this** como propietario para comportamiento modal correcto

Matriz de Validaciones Disponibles (v2.3.0)

#	Validación	Checkbox	Tipo	Restricción	Versión
1	Decimales (Enteros)	chkDecimales	Data Validation Custom	Sí ★	v2.3.0
2	Catálogos	chkCatalogos	Data Validation List	Sí	v2.2.2
3	NS	chkNS	Data Validation + Fórmula	Sí	v2.3.0
4	Formato Texto	chkFormatoTexto	Data Validation + Auxiliar	Sí ★	v2.3.0
5	Bloqueo Dinámico	chkBloqueo	Data Validation + Formatos	Sí	v2.2.4
6	Campos Vacíos	chkBlancos	Formato + Fórmula	No	v2.2.4
7	Especifique Clave	chkEspClave	Motor Auxiliar + Formato	No	v2.3.0 ★
8	Años	chkFechas	Data Validation Custom	Sí	v2.3.0 ★

Conclusión

SAVCNG ExcelDNA en versión 2.3.0 alcanza ocho tipos de validación complementarios. Las nuevas incorporaciones (EspClave y Años) junto con las mejoras a Decimales, NS y FormatoTexto refuerzan significativamente la integridad y la experiencia de captura en los formularios de censo.

Fortalezas de v2.3.0: ✓ Ocho tipos de validación complementarios

✓ Restricciones preventivas (el usuario no puede ingresar datos inválidos)

✓ Validaciones informativas para guía visual

✓ Coexistencia automática entre validaciones

✓ Gestión de memoria COM con `Marshal.ReleaseComObject`

✓ Compatibilidad universal multiidioma (`.Formula` en inglés)

✓ Interfaz completamente guiada por pasos

Recomendaciones para v3.0:

- Historial de auditoría de validaciones aplicadas
- Exportar/importar configuraciones en JSON
- Vista previa de fórmulas antes de aplicar
- Undo/Redo de validaciones
- Validaciones de fecha completa (día/mes/año)
- Caché de configuraciones frecuentes

Versión: 2.3.0.0

Última Actualización: 11 de julio de 2026

Estado: En Desarrollo Continuo

Rama de Git: Desarrollo